The Journal of Open Source Software

LightKrylov: Lightweight implementation of Krylov subspace techniques in modern Fortran

J. Simon Kern [1*], **Ricardo S. Frantz** [1*], and **Jean-Christophe Loiseau** [1*¶]

1 Arts et Métiers Institute of Technology ¶ Corresponding author * These authors contributed equally.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 20 October 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

LightKrylov is a Fortran package with minimal dependencies providing a collection of Krylov-based methods for solving linear systems, computing the leading eigenpairs of a linear operator as well as its leading singular triplets. When solving linear algebra problems, direct methods tend to have a computational cost scaling as $\mathcal{O}(n^3)$ making them impractical for very large-scale problems. Even when A is sparse, care needs to be taken when computing matrix factorizations since they can incur an $\mathcal{O}(n^2)$ storage cost in the worst case by introducing fill-in. In contrast, Krylov methods only need a function computing the matrix-vector product $u \leftarrow Av$ (or $u \leftarrow A^H v$) to iteratively construct a *Krylov subspace* (Krylov, 1931) from which the solutions can be extracted. We refer interested readers to Ipsen & Meyer (1998) for an introduction to Krylov methods, to Saad (2003) for technical details, and to Frantz et al. (2023) for examples in the field of computational fluid dynamics.

Statement of need

A collection of Krylov-based algorithms in pure modern Fortran

LightKrylov aims to provide Fortran users with familiar SciPy-inspired interfaces to widely used Krylov techniques. These include:

- **Krylov processes:** Arnoldi factorization, Golub-Kahan bidiagonalization, as well as Lanczos tridiagonalization for Hermitian operators.
- **Krylov methods:**
  - **Linear systems** - Conjugate Gradient (CG), Generalized Minimal Residual method (GMRES) and Flexible GMRES.
  - **Spectral decomposition** - Arnoldi iteration with Krylov-Schur restart for non-Hermitian operators, Lanczos tridiagonalization with thick restart for Hermitian ones.
  - **SVD** - restarted Golub-Kahan bidiagonalization.

Libraries exposing more methods exist in other languages, e.g., Krylov.jl (Montoison & Orban, 2023) in Julia or PETSc (Balay et al., 2015). Integrating multi-language libraries into existing Fortran codes often presents significant challenges related to build systems, dependency management, or potential performance overhead from language interfacing ("two-language problem"). In contrast, LightKrylov provides a pure Fortran alternative, compliant with the Fortran 2018 standard, and only requires the Fortran standard library stdlib as an external runtime dependency. It leverages stdlib not only for basic utilities but specifically builds upon stdlib_linalg for foundational linear algebra operations, ensuring adherence to community standards and benefiting from its rapid, ongoing development.

Additionally, using fypp (Aradi, 2023) as a build-time dependency significantly reduces code duplication and maintenance effort, ensuring consistency across different data types (single,

double, extended, and quadruple precision) both for real and complex numbers while keeping the core algorithmic logic centralized. fypp is a Python-based Fortran preprocessor automatically generating the necessary code for different data types. Finally, its build process relies on the Fortran package manager fpm, greatly facilitating its installation and integration with the modern Fortran ecosystem.

Licensing: LightKrylov is distributed under the permissive BSD 3-Clause License, encouraging broad adoption and contribution.

Testing/CI: The library includes a suite of unit and integration tests executed via GitHub Actions. The set of compilers currently tested includes gfortran (versions 12–14) and the Intel ifort and ifx compilers. The operating systems include the latest Ubuntu, Windows, and macOS 13.

A focus on abstract linear operators and abstract vectors

Krylov methods can be implemented without explicit reference to the particular data structure used to represent a vector or a linear operator, nor to how the actual matrix-vector product is implemented. To do so, LightKrylov uses modern Fortran abstract type constructs. A stripped-down version of the abstract vector type is shown below.

```fortran
type, abstract :: abstract_vector_rdp
  contains
    ! Abstract procedure to compute the scalar-vector product.
    procedure(abstract_scal_rdp), pass(self), deferred :: scal
    ! Abstract procedure to compute y = alpha*x + beta*y.
    procedure(abstract_axpby_rdp), pass(self), deferred :: axpby
    ! Abstract procedure to compute the vector dot product.
    procedure(abstract_dot_rdp), pass(self), deferred :: dot
end type
```

These type-bound procedures cover the basic operations on vectors: scalar-vector product, linear combination of two vectors, and the dot product. These operations are the essential building blocks required by Krylov algorithms. The signatures of these type-bound procedures follow, to the extent possible, the standard signatures of the corresponding BLAS functions for a more familiar use. For instance, the abstract_axpby_rdp interface reads

```fortran
abstract interface
  subroutine abstract_axpby_rdp(alpha, vec, beta, self)
    double precision, intent(in)    :: alpha, beta
    class(abstract_vector_rdp), intent(in)    :: vec
    class(abstract_vector_rdp), intent(inout) :: self
  end subroutine
end interface
```

mimicking the signature of the (extended) BLAS-1 subroutine axpby. In practice, LightKrylov requires the definition of two additional procedures, one to set a vector to zero and the second to fill it with random data.

Similarly, a stripped-down version of an abstract linear operator type is shown below.

```fortran
type, abstract :: abstract_linop_rdp
  contains
    ! Abstract procedure to compute the matrix-vector product y = A * x
    procedure(abstract_matvec_rdp), pass(self), deferred :: matvec
    ! Abstract procedure to compute the matrix-vector product y = A^H * x
    procedure(abstract_matvec_rdp), pass(self), deferred :: rmatvec
end type
```

The two type-bound procedures need to implement the matrix-vector product and the transposed matrix-vector product. The corresponding abstract interface reads

```
abstract interface
  subroutine abstract_matvec_rdp(self, vec_in, vec_out)
    class(abstract_linop_rdp), intent(inout) :: self
    class(abstract_vector_rdp), intent(in)    :: vec_in
    class(abstract_vector_rdp), intent(out)   :: vec_out
  end subroutine
end interface
```

Using such `abstract` types enables us to focus on the high-level implementation of the different Krylov-based algorithms while leaving the performance-critical details of the vector and matrix-vector operations to the users. After extending these `abstract` types for their particular applications, users can solve linear systems or compute eigenvalues as easily as

```
! Solve linear system.
call gmres(A, b, x, info)

! Compute eigenvalues and eigenvectors.
call eigs(A, V, lambda, residuals, info)
```

In addition, `LightKrylov` exposes abstract interfaces to define preconditioners, as well as a Newton-GMRES solver to find the roots of a multivariate function $f : \mathbb{K}^n \to \mathbb{K}^n$ (where $\mathbb{K}$ is a placeholder for $\mathbb{R}$ or $\mathbb{C}$). Examples illustrating how to extend these `abstract` types and interfaces for computing the leading eigenpairs of the linearized Ginzburg-Landau equation or computing unstable periodic orbits of the Rössler system and studying their Lyapunov exponents can be found here.

Performance in a production-ready open-source codebase

`LightKrylov` was successfully integrated into neklab, a toolbox for stability and bifurcation analysis using the high-performance spectral element solver `Nek5000`. The abstract vector interface allows direct use of `Nek5000`'s distributed data structures, and the pure-Fortran nature facilitated integration with its existing build system, demonstrating the library's suitability for large-scale HPC applications.

Hydrodynamic stability of an unstable fixed point of the nonlinear Navier-Stokes equations

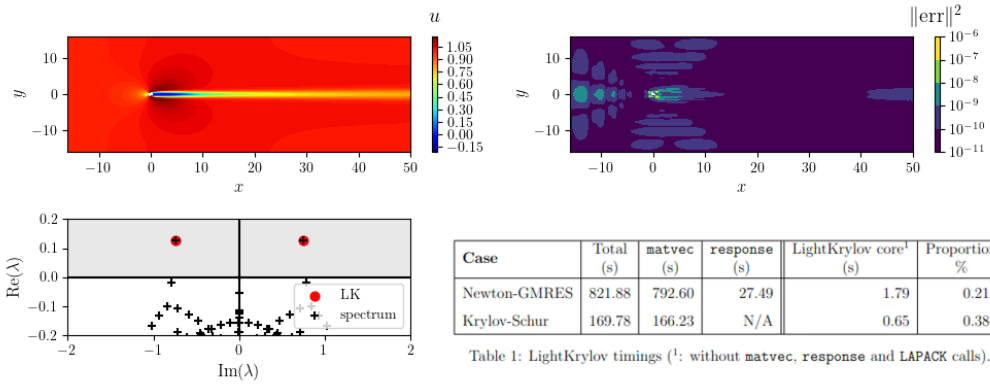
Computing (stable or unstable) fixed points of the nonlinear governing equations is a necessary step when studying the stability properties of a nonlinear system. In hydrodynamic instability applications, this amounts to finding a stationary solution of the nonlinear Navier-Stokes equations. Investigating the spectral properties of the linearized Navier-Stokes operator about such fixed points is critical to understand the transition to unsteadiness and turbulence in many flow applications.

Using the supercritical two-dimensional flow past a circular cylinder at Reynolds number of 100 as an example, we showcase the efficient integration of `LightKrylov` and `Nek5000` and validate the results with algorithms provided by the `KTH Framework` toolbox (Massaro et al., 2024) based on the same Navier-Stokes solver. In all cases, the governing equations are discretized with 1996 spectral elements using a 5th-order polynomial approximation in each direction, resulting in 71,856 grid points and roughly 175,000 degrees of freedom (i.e., the two velocity components and pressure). All computations were run in parallel on 12 Intel Core Ultra 7 processors and the numerical settings, in particular those internal to `Nek5000`, are identical for all runs.

The unstable fixed point of the nonlinear Navier-Stokes equations is obtained using both `LightKrylov`'s *time-stepper*-based Newton-GMRES solver (Frantz et al., 2023) and the `KTH Framework` implementation of the selective frequency damping algorithm (Åkervik et al., 2006) (often considered as the gold standard in this community). The top row in Figure 1 depicts the streamwise (u) velocity distribution at the fixed point (left) and the pointwise squared difference between the solutions obtained with the two solution techniques and a tolerance of $\varepsilon = 10^{-8}$ (right). Note that, while the definition of the error norm slightly differs between the methods, the one used by the Newton-GMRES solver is stricter. The code to run this example can be found here.

For the linear stability analysis, the first complex-conjugate eigenpair is computed with the Krylov-Schur algorithm using `LightKrylov`'s abstract interfaces. In the lower-left panel of Figure 1, it is compared to the spectrum computed using `KTH Framework` relying under the hood on `ARPACK`. Both solvers effectively use a *time-stepper* approach, approximating the leading eigenvalues of the exponential propagator $\exp(\tau \mathbf{A})$ rather than $\mathbf{A}$ directly. The code to run this example can be found here.

The table in the lower-right panel of Figure 1 summarizes the wall-clock times of the `neklab` computations. Isolating the intrinsic cost of the algorithms in `LightKrylov` from the cost of the calls to LAPACK and the linear and nonlinear Navier-Stokes solvers (`matvec` and `response`, respectively) conclusively shows that the use of extended `abstract` types and object-oriented programming in Fortran incurs a negligible computational overhead for such large-scale applications.



Case	Total (s)	matvec (s)	response (s)	LightKrylov core[1] (s)	Proportion %
Newton-GMRES	821.88	792.60	27.49	1.79	0.218
Krylov-Schur	169.78	166.23	N/A	0.65	0.384

Table 1: LightKrylov timings ([1]: without `matvec`, `response` and `LAPACK` calls).

Figure 1: Validation of `LightKrylov`: Newton-GMRES and `eigs`.

Perspectives

Despite being in its early development stage, `LightKrylov` has already been used in hydrodynamic stability production runs on dozens of processors. It has also been interfaced with neko, a modernized implementation of `Nek5000` running on GPUs, and is currently being interfaced with `dNami`, a high-performance source code generator for hyperbolic partial differential equations.

Ongoing development efforts include:

- **Build system:** `LightKrylov` relies on the Fortran package manager `fpm` for its build. While it ensures nice integration into the modern Fortran ecosystem, it also limits its usage to codes built using `fpm` as well. An additional build system based on `CMake` for easier integration into non-`fpm` codes will be proposed shortly.
- **Krylov processes:** As of summer 2025, `LightKrylov` provides implementations for three of the most important Krylov processes. Future releases will extend this list by including the non-Hermitian Lanczos and two-sided Arnoldi factorizations useful for reduced-order

modeling of linear time-invariant (LTI) systems as well as the Saunders-Simon-Yip process (Saunders et al., 1988).

- **Iterative solvers:** Beyond `CG` and `GMRES`, the integration of new Krylov processes will enable us to provide an extended list of iterative solvers, including `MINRES` (indefinite Hermitian systems), `CGNE` and `LSQR` (least-squares problems), and `LSMR` (least-norm problems).

Finally, the development of `LightKrylov` spurred a community-driven effort in the fortran-lang ecosystem to integrate some of its features (namely the abstraction layer and the iterative solvers) into the Fortran standard library `stdlib`.

Acknowledgements

We acknowledge the financial support of the French National Agency for Research (ANR) through the ANR-33-CE46-0008-CONMAN grant agreement. We would also like to thank the fortran-lang community for the development of `stdlib`, and in particular Frederico Perini, Jeremie Vandenplas, and Jose Alvez for their work on the `stdlib_linalg` module which `LightKrylov` relies on.

References

Åkervik, E., Brandt, L., Henningson, D. S., Hœpffner, J., Marxen, O., & Schlatter, P. (2006). Steady solutions of the navier-stokes equations by selective frequency damping. *Physics of Fluids*, *18*(6), 068102. https://doi.org/10.1063/1.2211705

Aradi, B. (2023). *fypp web page*. https://github.com/aradi/fypp

Balay, S., Abhyankar, S., Adams, M. F., Brown, J., Brune, P., Buschelman, K., Dalcin, L., Eijkhout, V., Gropp, W. D., Kaushik, D., Knepley, M. G., McInnes, L. C., Rupp, K., Smith, B. F., Zampini, S., & Zhang, H. (2015). *PETSc web page*. http://www.mcs.anl.gov/petsc

Frantz, R. A. S., Loiseau, J.-Ch., & Robinet, J.-Ch. (2023). Krylov Methods for Large-Scale Dynamical Systems: Application in Fluid Dynamics. *Applied Mechanics Reviews*, *75*(3).

Ipsen, I. C. F., & Meyer, C. D. (1998). The Idea Behind Krylov Methods. *The American Mathematical Monthly*, *105*(10), 889–899.

Krylov, A. N. (1931). On the numerical solution of the equation by which, in technical matters, frequencies of small oscillations of material systems are determined. *Izvestija AN SSSR (News of Academy of Sciences of the USSR), Otdel. Mat. I Estest. Nauk*, *7*(4), 491–539.

Massaro, D., Peplinski, A., Stanly, R., Mirzareza, S., Lupi, V., Mukha, T., & Schlatter, P. (2024). A comprehensive framework to enhance numerical simulations in the spectral-element code Nek5000. *Computer Physics Communications*, *302*, 109249. https://doi.org/10.1016/j.cpc.2024.109249

Montoison, A., & Orban, D. (2023). Krylov.jl: A julia basket of hand-picked krylov methods. *Journal of Open Source Software*, *8*(89), 5187.

Saad, Y. (2003). *Iterative Methods for Sparse Linear Systems*. Society for Industrial and Applied Mathematics. ISBN: 978-0-89871-534-7

Saunders, M. A., Simon, H. D., & Yip, E. L. (1988). Two conjugate-gradient-type methods for unsymmetric linear equations. *SIAM Journal on Numerical Analysis*, *25*(4), 927–940. https://doi.org/10.1137/0725052

DRAFT